

Section 1 / More About `ldr`

Overview

In this chapter we examine the difference between loading the address of (a pointer to) a data label versus loading the data at the label. Both use the `ldr` instruction however, the assembler (on Linux) actually does some trickery behind the scenes to accomplish the loads.

Note - this chapter describes `ldr` from the perspective of Linux. For a brief discussion of how Apple Mac OS avoids using `ldr` to load the address of labels, please see [here](#) and [below](#).

Length of Instructions

All AARCH64 instructions are 4 bytes in width.

Length of Pointers

All AARCH64 pointers are 8 bytes in width[†].

[†]While this is technically true, typically only the lower 39, 42 or 48 bits of addresses in Linux systems are used - i.e. the virtual address space of an ARM Linux process is smaller than 64 bits. The upper bits are set to zero when considering the address as an 8-byte value.

How to Specify an Address Too Big to Fit in an Instruction?

The title of this section sets the table for the need for trickery. All labels refer to addresses. Addresses are 8 bytes[†] in width but all instructions are 4 bytes in width. Clearly, we cannot fit the full address of a label in an instruction.

Some ISAs (not ARM) have variable length instructions. The instruction may be four bytes wide but it tells the CPU that the next eight bytes are an operand of the instruction. Thus the true instruction width is 12 bytes. This is not true of the ARM ISA.

All instructions are 4 bytes wide. All of them.

“`ldr x_register, =label`” is a Pseudo Instruction

When you assemble an instruction looking like:

```
ldr    x1, =label
```

the assembler puts the address of the label into a special region of memory called a “literal pool.” What matters is this region of memory is placed immediately after (therefore nearby) your code.

Then, the assembler computes the difference between the address of the current instruction (the `ldr` itself) and the address of the data in the literal pool made from the labeled data.

The assembler generates a different `ldr` instruction which uses the difference (or offset) of the data relative to the program counter (`pc`). The `pc` is non-other the address of the current instruction.

Because the literal pool for your code is located nearby your code, the offset from the current instruction to the data in the pool is a relatively **small** number. Small enough, to fit inside a four byte `ldr` instruction.

```
ldr    x1, [pc, offset to data in literal pool]
```

Example Program for Demonstrating Use of Literal Pool

Here is a sample program demonstrating the difference between:

```
ldr    x1, =q
```

and

```
ldr    x1, q
```

Note the difference is that the first has an `=` sign before the label and the second does not.

Also note, that when line 15 is executed, the program will **crash**.

```

.global    main                                // 1
.text                                           // 2
.align     2                                  // 3
// 4
main:      str        x30, [sp, -16]!           // 5
// 6
ldr        x0, =fmt                            // Loads the address of fmt // 7
ldr        x1, =q                               // Loads the address of q   // 8
ldr        x2, [x1]                             // Loads the value at q         // 9
bl         printf                               // Calls printf()              // 10
// 11
// 12
ldr        x0, =fmt                            // Loads the address of fmt // 13
ldr        x1, q                               // Loads the VALUE at q       // 14
ldr        x2, [x1]                             // CRASH!                     // 15
bl         printf                               // 16
// 17
ldr        x30, [sp], 16                       // 18
mov        w0, wzr                             // 19
ret                                              // 20
// 21
```

```

        .data                                // 22
q:      .quad      0x1122334455667788        // 23
fmt:    .asciz     "address: %p value: %lx\n" // 24
                                           // 25
        .end                                // 26
                                           // 27

```

Disassembling the binary machine code of the executable generated with the above source code will include:

```

00000000000007a0 <main>:
7a0: f81f0ffe str x30, [sp, #-16]!
7a4: 58000160 ldr x0, 7d0 <main+0x30>
7a8: 58000181 ldr x1, 7d8 <main+0x38>
7ac: f9400022 ldr x2, [x1]
7b0: 97ffffb4 bl 680 <printf@plt>
7b4: 580000e0 ldr x0, 7d0 <main+0x30>
7b8: 580842c1 ldr x1, 11010 <q>
7bc: f9400022 ldr x2, [x1]
7c0: 97ffffb0 bl 680 <printf@plt>
7c4: f84107fe ldr x30, [sp], #16
7c8: 2a1f03e0 mov w0, wzr
7cc: d65f03c0 ret

```

and

```

0000000000011010 <q>:
11010: 55667788
11014: 11223344

```

Let's examine the second snippet first.

It says 0000000000011010 <q>:. This means that what comes next is the data corresponding to what is labeled `q` in our source code. Notice the relocatable address of 11010. We will explain "relocatable address" below.

Now, look at the disassembled code on the line beginning with `7b8`. It reads `ldr x1, 11010`. So the disassembled executable is saying "go to address 11010 and fetch its contents" which are our 1122334455667788.

This is not the whole story.

Relocation of Addresses When Executing

None of the addresses we have seen so far are the final addresses that will be used once the program is actually running. All addresses will be *relocated*.

One reason for this is a guard against malware. A technique called Address Space Layout Randomization (ASLR) prevents malware writers from being able

to know ahead where to modify your executable in order to accomplish their nefarious purposes.

This image shows `gdb` in `layout regs` at the time our program is loaded.

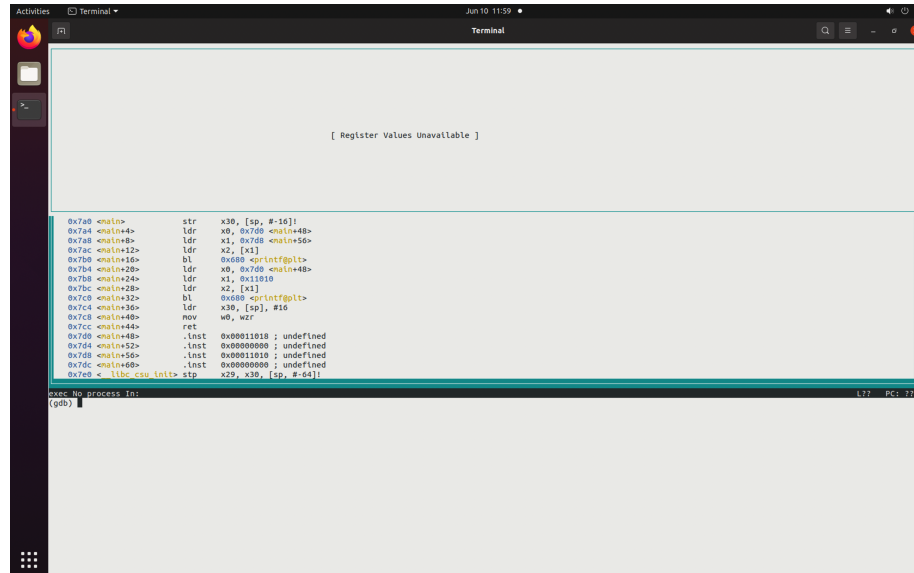


Figure 1: Prior to Launch

Notice that all of the addresses match the disassemblies given above. For example `main()` starts at `7a0`.

Now watch what happens the the program is actually launched (see Figure 2):

Suddenly all the address change to much larger values.

In fact, the addresses all seem to be six bytes long!

Why are these addresses only six bytes long when all pointers are 8 bytes long?

Sixty four bit ARM Linux kernels allocate 39, 42 or 48 bits for the size of a process's virtual address space. Notice 42 and 48 bit values require 6 bytes to hold them. A virtual address space is all of the addresses a process can generate / use. Further, all addresses used by processes are virtual addresses.

Kernels supporting other VA spaces, including 52 bit address spaces are possible but less common.

The salient point is that even six bytes is far too large to fit in a four byte instruction. GDB is masking the pseudo instruction and showing what the effective addresses are.**

```

Register group: general
x0 0x1 187649984472992 x1 0xffffffffff048 281474976706632 x2 0xfffffffff08 281474976706648
x3 0xaaaaaaaaa7a0 187649984472992 x4 0x0 0 x5 0x12852ca05254e7 131323840057842919
x6 0xfffff7b7b10 281474842196752 x7 0x4010000100004000 4616189622349742880 x8 0xffffffffffffff -1
x9 0xffff 4095 x10 0x0 0 x11 0x0 0
x12 0xfffff7a4b4e48 281474840706632 x13 0x0 0 x14 0x0 0
x15 0xfffff7a47 1879848007 x16 0x1 1 x17 0xfffff7e6828 281474840025128
x18 0x73532540 1934713408 x19 0xaaaaaaaaa7e0 187649984473856 x20 0x0 0
x21 0xaaaaaaaaa590 187649984472720 x22 0x0 0 x23 0x0 0
x24 0x0 0 x25 0x0 0 x26 0x0 0
x27 0x0 0 x28 0x0 0 x29 0xfffffffef0 281474976706288
x30 0xfffffffef0 281474840025128 sp 0xfffffffef0 0xfffffffef0 pc 0xaaaaaaaaa7a0 0
cpsr 0x00000000 [ EL=0 S=0 C=Z ] pauth_dnask 0x7f000000000000 pauth_cnask 0x7f000000000000 fpcr 0x0 0

Disassemble: 28335 In: main
(gdb) b main
Breakpoint 1 at 0x7a0: file ldr_tests.s, line 5.
(gdb) run
Starting program: /media/psf/Home/Documents/asn_book/section_1/regs/a.out
Breakpoint 1, main () at ldr_tests.s:5
(gdb)

```

Figure 2: After breakpoint and launch

Now let's step forward to see the results of the first `ldr` of the `printf()` template / format string into `x0`. See Figure 3.

There is a pointer in `x0` ending in `b018`. Notice this is **NOT** the value encoded in the instruction ending in `a7d0`. This is our only indirect evidence that the instruction we wrote has been modified to use some calculated offset from the `pc`.

To finish, here is how we confirm `x0` is indeed correct. See Figure 4.

Notice down below the `x/s $x0` prints the value in memory corresponding to the address contained in `x0`.

Finally see Figure 5.

At the outset of this discussion we said that this program will crash on source code line 15. See if you can work out why. Take a moment before reading further.

Now that you have a hypothesis in mind, take a look at this screenshot showing the state of `x1` after this instruction: `ldr x1, q` is executed. See Figure 6.

Notice that what is in `x1` this time looks very different from the previous attempt at printing. Notice still more that the value now in `x1` is the value of `q`, not its address.

Naturally, the next instruction which tries to dereference the value of `q` rather than its address, causes a crash. See Figure 7.

```

Register group: general
x0 0x00000000 187649984540096 x1 0xfffffffff048 281474976706632 x2 0xfffffffff058 281474976706648
x3 0xaaaaaaaa7a0 187649984472992 x4 0x0 0 0 x5 0x128825ca60254e7 133535840857842919
x6 0xffffffff7b7b10 281474842196752 x7 0x4010000100004000 4616189622349742880 x8 0xffffffffffffff -1
x9 0xffff 4095 x10 0x0 0 x11 0x0 0
x12 0xffffffff4b4e48 281474840766632 x13 0x0 0 x14 0x0 0
x15 0xffffffff7 1879680807 x16 0x1 1 x17 0xffffffffc68d28 281474840825128
x18 0x73516240 1934713408 x19 0xaaaaaaaa7e0 187649984473056 x20 0x0 0
x21 0xaaaaaaaaa690 187649984472720 x22 0x0 0 x23 0x0 0
x24 0x0 0 x25 0x0 0 x26 0x0 0
x27 0x0 0 x28 0x0 0 x29 0xfffffffffe0 281474976706288
x30 0xffffffffe0e10 281474840825360 sp 0xfffffffffe0 0
cpsr 0x60201000 [ EL0 S8S SS C Z ] fpcr 0x0 0
pauth_dnask 0x7f000000000000 35747322042253312 pauth_cnask 0x7f000000000000 35747322042253312

0x7a0: <main> str x30, [sp, #16]!
0x7a0: <main> ldr x0, [x0, #16]!
0x7a0: <main> ldr x1, [x0, #16]!
0x7a0: <main> ldr x2, [x1]
0x7a0: <main> bl 0xaaaaaaaa800 <printf@plt>
0x7a0: <main> ldr x0, [x0, #16]!
0x7a0: <main> ldr x1, [x0, #16]!
0x7a0: <main> ldr x2, [x1]
0x7a0: <main> bl 0xaaaaaaaa800 <printf@plt>
0x7a0: <main> ldr x30, [sp, #16]
0x7a0: <main> mov w0, w2r
0x7a0: <main> ret
0x7a0: <main> orn x24, x0, x11, asr #44
0x7a0: <main> .inst 0x00000000; undefined
0x7a0: <main> orn x10, x0, x11, asr #44
0x7a0: <main> .inst 0x00000000; undefined
0x7a0: <main> stp x29, x30, [sp, #64]!
0x7a0: <main> .libc.csu .int

native process 28535 in: main
(gdb) b main
Breakpoint 1 at 0x7a0: file ldr_tests.s, line 5.
(gdb) run
Starting program: /media/psf/Home/Documents/asn_book/section_1/regs/a.out
Breakpoint 1, main () at ldr_tests.s:5
(gdb) n
(gdb) n
(gdb)

```

Figure 3: Results of first ldr

```

Register group: general
x0 0x00000000 187649984540096 x1 0xfffffffff048 281474976706632 x2 0xfffffffff058 281474976706648
x3 0xaaaaaaaa7a0 187649984472992 x4 0x0 0 0 x5 0x128825ca60254e7 133535840857842919
x6 0xffffffff7b7b10 281474842196752 x7 0x4010000100004000 4616189622349742880 x8 0xffffffffffffff -1
x9 0xffff 4095 x10 0x0 0 x11 0x0 0
x12 0xffffffff4b4e48 281474840766632 x13 0x0 0 x14 0x0 0
x15 0xffffffff7 1879680807 x16 0x1 1 x17 0xffffffffc68d28 281474840825128
x18 0x73516240 1934713408 x19 0xaaaaaaaa7e0 187649984473056 x20 0x0 0
x21 0xaaaaaaaaa690 187649984472720 x22 0x0 0 x23 0x0 0
x24 0x0 0 x25 0x0 0 x26 0x0 0
x27 0x0 0 x28 0x0 0 x29 0xfffffffffe0 281474976706288
x30 0xffffffffe0e10 281474840825360 sp 0xfffffffffe0 0
cpsr 0x60201000 [ EL0 S8S SS C Z ] fpcr 0x0 0
pauth_dnask 0x7f000000000000 35747322042253312 pauth_cnask 0x7f000000000000 35747322042253312

0x7a0: <main> str x30, [sp, #16]!
0x7a0: <main> ldr x0, [x0, #16]!
0x7a0: <main> ldr x1, [x0, #16]!
0x7a0: <main> ldr x2, [x1]
0x7a0: <main> bl 0xaaaaaaaa800 <printf@plt>
0x7a0: <main> ldr x0, [x0, #16]!
0x7a0: <main> ldr x1, [x0, #16]!
0x7a0: <main> ldr x2, [x1]
0x7a0: <main> bl 0xaaaaaaaa800 <printf@plt>
0x7a0: <main> ldr x30, [sp, #16]
0x7a0: <main> mov w0, w2r
0x7a0: <main> ret
0x7a0: <main> orn x24, x0, x11, asr #44
0x7a0: <main> .inst 0x00000000; undefined
0x7a0: <main> orn x10, x0, x11, asr #44
0x7a0: <main> .inst 0x00000000; undefined
0x7a0: <main> stp x29, x30, [sp, #64]!
0x7a0: <main> .libc.csu .int

native process 28535 in: main
(gdb) b main
Breakpoint 1 at 0x7a0: file ldr_tests.s, line 5.
(gdb) run
Starting program: /media/psf/Home/Documents/asn_book/section_1/regs/a.out
Breakpoint 1, main () at ldr_tests.s:5
(gdb) n
(gdb) n
(gdb) x/s $x0
0xaaaaaaaa7a0: "address: 0x value: 0x\n"
(gdb)

```

Figure 4: Confirming x0



We have learned how the addresses corresponding to labels can be found. We also have learned how the contents of memory at those labels can be retrieved.

In both cases, the assembler will likely do some magical translation of your simple `ldr` instruction into something involving offsets so that the resulting offset can fit into an instruction where the full address cannot.

Questions

8

Apple Silicon

You’ve seen that under Linux, there is a pseudo-instruction which hides some trickery:

```
ldr    x0, =label
```

This works if `label` is +/- four mebibytes (as megabytes are now called) away from the `ldr` instruction.

A downside of this approach is that the literal pool, from which the address is loaded, resides in RAM. This means each of these `ldr` pseudo instructions incurs a memory reference.

Apple “thinks different.” The above instruction will not pass the assembler on a Mac OS machine. Instead, Apple uses two techniques which can access labels no matter where they are *without incurring a reference to memory*.

Apple accomplishes this by splitting the loading of the address of a label into two instructions. The first causes the base address of the *page* on which the label resides to be loaded. The page number is calculated for you based upon the label. The second adds to the base address, the offset in the page at which the label can be found.

Both of these values are computed at build time and therefore do not need to reference memory. This is a good thing.

Here is how one would load the address of a label which is outside the source code module:

```
.macro  GLD_ADDR    xreg, label    // Get a global address
#if defined(__APPLE__)
        adrp        \xreg, _\label@GOTPAGE
        add         \xreg, \xreg, _\label@GOTPAGEOFF
#else
        ldr         \xreg, =\label
#endif
.endm
```

This is a macro from our Apple Linux Convergence Suite.

It shows how, on Apple systems, the higher bits of the address is loaded from the starting address of the page on which the symbol sits and then the remainder (the offset) is added in.

The `G` in `GLD_ADDR` stands for global.

If the label is defined in the same source code module:

```
.macro  LLD_ADDR xreg, label
#if defined(__APPLE__)
        adrp        \xreg, \label@PAGE
```

```

        add        \xreg, \xreg, \label@PAGEOFF
#else
        ldr        \xreg, =\label
#endif
.endm

```

The difference being @PAGE versus @GOTPAGE, etc.

The first L in LLD_ADDR stands for local.

Avoiding the Memory Reference on Linux

The technique Apple uses for loading the address of labels can be used on Linux as well so as to avoid the reference to memory (literal pool).

Suppose `s` is a locally defined symbol. Then:

```

adrp    x0, s
add     x0, x0, :lo12:s

```

duplicates the approach Apple uses.